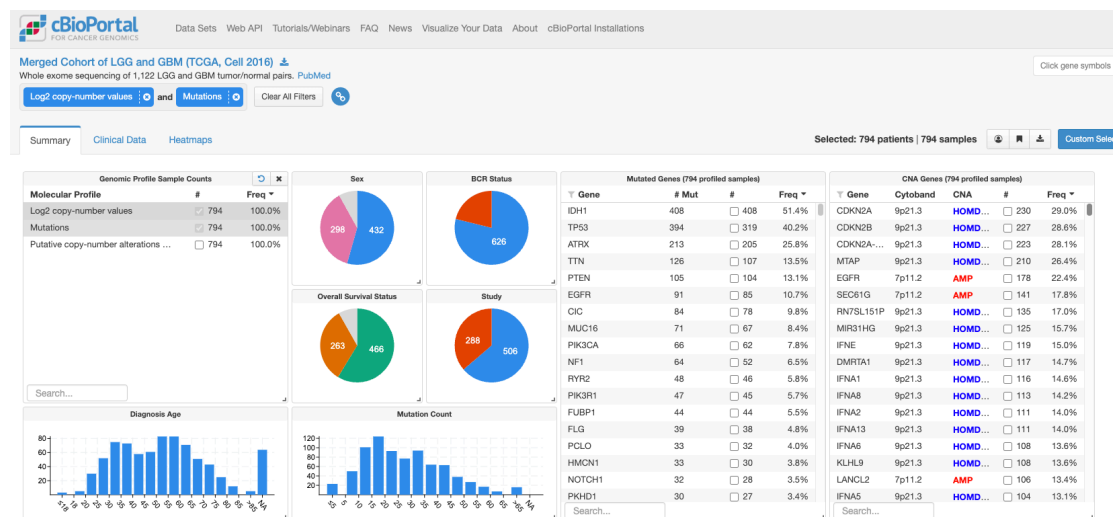


```
In [1]: import os
import flexynesis
import torch
import numpy as np
import seaborn as sns
import pandas as pd
import random
import lightning as pl
```

Seed set to 42

Finding Survival Markers in Lower Grade Glioma (LGG) and Glioblastoma Multiforme (GBM)seed



Here, we demonstrate the capabilities of `flexynesis` on a multi-omic dataset of 506 Brain Lower Grade Glioma (LGG) and 288 Glioblastoma Multiforme (GBM) samples with matching mutation and copy number alteration data downloaded from the [cbioportal](#). The data was split into `train` (70% of the samples) and `test` (30% of the samples) data folders. The data files were processed to follow the same nomenclature.

- `cna.csv` contains "copy number alteration" data
- `mut.csv` contains "mutation" data, which is a binary matrix of genes versus samples.
- `clin.csv` contains "clinical/sample metadata", which is a table of clinical parameters such as age, sex, disease type, histological diagnosis, and overall survival time and status.

Data Download

The data can be downloaded as follows:

```
In [2]: if not os.path.exists("lgggbm_tcga_pub_processed"):
        !wget -O lgggbm_tcga_pub_processed.tgz "https://bimsbstatic.mdc-berlin.de/akalin/buyar/flexynesis-benchmark-datasets/lgggbm_tcga_pub_processed.tgz"
--2025-03-14 09:32:24-- https://bimsbstatic.mdc-berlin.de/akalin/buyar/flexynesis-benchmark-datasets/lgggbm_tcga_pub_processed.tgz
141.80.181.47, 141.80.181.46rlin.de (bimsbstatic.mdc-berlin.de)...
connected. to bimsbstatic.mdc-berlin.de (bimsbstatic.mdc-berlin.de)|141.80.181.47|:443...
HTTP request sent, awaiting response... 200 OK
Length: 1817945 (1.7M) [application/octet-stream]
Saving to: 'lgggbm_tcga_pub_processed.tgz'

lgggbm_tcga_pub_pro 100%[=====>] 1.73M 3.57MB/s in 0.5
s

2025-03-14 09:32:25 (3.57 MB/s) - 'lgggbm_tcga_pub_processed.tgz' saved [1817945/1817945]

lgggbm_tcga_pub_processed/
lgggbm_tcga_pub_processed/train/
lgggbm_tcga_pub_processed/train/mut.csv
lgggbm_tcga_pub_processed/train/cna.csv
lgggbm_tcga_pub_processed/train/clin.csv
lgggbm_tcga_pub_processed/test/
lgggbm_tcga_pub_processed/test/mut.csv
lgggbm_tcga_pub_processed/test/cna.csv
lgggbm_tcga_pub_processed/test/clin.csv
```

Importing Train and Test Datasets

We import train and test datasets including mutations and copy number alterations.

We rank genes by Laplacian Scores and pick top 10% of the genes, while removing highly redundant genes with a correlation score threshold of 0.8 and a variance threshold of 50%. By setting `concatenate` to `False`, we will be doing an `intermediate` fusion of omic layers.

```
In [3]: data_importer = flexynesis.DataImporter(path='lgggbm_tcga_pub_processed'
        data_types = ['mut', 'cna'], log_
        concatenate=False, top_percentile
        min_features=1000, correlation_th
        variance_threshold=0.5)
train_dataset, test_dataset = data_importer.import_data()
```

```

[INFO] ===== Importing Data =====
[INFO] Validating data folders...

[INFO] ----- Reading Data -----
[INFO] Importing lgggbm_tcga_pub_processed/train/clin.csv...
[INFO] Importing lgggbm_tcga_pub_processed/train/cna.csv...
[INFO] Importing lgggbm_tcga_pub_processed/train/mut.csv...

[INFO] ----- Reading Data -----
[INFO] Importing lgggbm_tcga_pub_processed/test/clin.csv...
[INFO] Importing lgggbm_tcga_pub_processed/test/cna.csv...
[INFO] Importing lgggbm_tcga_pub_processed/test/mut.csv...

[INFO] ----- Checking for problems with the input data -----
[INFO] Data structure is valid with no errors or warnings.

[INFO] ----- Processing Data (train) -----

[INFO] ----- Cleaning Up Data -----

[INFO] working on layer: mut
[INFO] Number of NA values: 0
[INFO] DataFrame mut - Removed 5561 features.

[INFO] working on layer: cna
[INFO] Number of NA values: 0
[INFO] DataFrame cna - Removed 12375 features.
[INFO] DataFrame mut - Removed 0 samples (0.00%).
[INFO] DataFrame cna - Removed 0 samples (0.00%).
[INFO] Implementing feature selection using laplacian score for layer: mut
with 5503 features and 556 samples
Calculating Laplacian scores: 100%|██████████| 5503/5503 [00:00<00:00, 15443.9
1it/s]
Filtering redundant features: 100%|██████████| 1000/1000 [00:00<00:00, 24082.7
7it/s]
[INFO] Implementing feature selection using laplacian score for layer: cna
with 12371 features and 556 samples
Calculating Laplacian scores: 100%|██████████| 12371/12371 [00:04<00:00, 3005.6
8it/s]
Filtering redundant features: 100%|██████████| 1237/1237 [00:00<00:00, 179254.9
1it/s]

```

```
[INFO] ----- Processing Data (test) -----
[INFO] ----- Cleaning Up Data -----

[INFO] working on layer:  mut
[INFO] Number of NA values:  0
[INFO] DataFrame mut – Removed 5627 features.

[INFO] working on layer:  cna
[INFO] Number of NA values:  0
[INFO] DataFrame cna – Removed 12382 features.
[INFO] DataFrame mut – Removed 0 samples (0.00%).
[INFO] DataFrame cna – Removed 0 samples (0.00%).

[INFO] ----- Harmonizing Data Sets -----

[INFO] ----- Finished Harmonizing -----

[INFO] ----- Normalizing Data -----

[INFO] ----- Normalizing Data -----
[INFO] Training Data Stats:  {'feature_count in: mut': 316, 'feature_count
in: cna': 1237, 'sample_count': 556}
[INFO] Test Data Stats:  {'feature_count in: mut': 316, 'feature_count in:
cna': 1237, 'sample_count': 238}
[INFO] Merging Feature Logs...
[INFO] Data import successful.
```

1. Exploratory Data Analysis

Before building any machine learning models on the data, it is important to first familiarize yourself with the data you are working with. It is important to know the available data matrices, their sizes/shapes, available clinical variables and how they are distributed.

Below you are asked to do simple explorations of the available data.

1.1 Print the shapes of the available data matrices

- How many features and samples are available per data type in train/test datasets?

```
In [5]: print(dir(train_dataset))
```

```
['__add__', '__annotations__', '__class__', '__class_getitem__', '__delatt
r__', '__dict__', '__dir__', '__doc__', '__eq__', '__format__', '__ge__',
 '__getattr__', '__getitem__', '__getstate__', '__gt__', '__hash__',
 '__init__', '__init_subclass__', '__le__', '__len__', '__lt__', '__module_
__', '__ne__', '__new__', '__orig_bases__', '__parameters__', '__reduce__',
 '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__slots__', '__
str__', '__subclasshook__', '__weakref__', '_is_protocol', 'ann', 'dat',
'feature_ann', 'features', 'get_dataset_stats', 'get_feature_subset', 'lab
el_mappings', 'samples', 'subset', 'variable_types']
```

```
In [6]: print(type(train_dataset))
        print(train_dataset)
```

```
<class 'flexynesis.data.MultiOmicDataset'>
<flexynesis.data.MultiOmicDataset object at 0xffffeff845590>
```

```
In [8]: print(type(test_dataset))
        print(test_dataset)
```

```
<class 'flexynesis.data.MultiOmicDataset'>
<flexynesis.data.MultiOmicDataset object at 0xfffff8c161650>
```

```
In [11]: print("Train Dataset Feature Count:", len(train_dataset.features))
          print("Train Dataset Sample Count:", len(train_dataset.samples))

          print("Test Dataset Feature Count:", len(test_dataset.features))
          print("Test Dataset Sample Count:", len(test_dataset.samples))
```

```
Train Dataset Feature Count: 2
Train Dataset Sample Count: 556
Test Dataset Feature Count: 2
Test Dataset Sample Count: 238
```

```
In [12]: print(train_dataset.get_dataset_stats())
          print(test_dataset.get_dataset_stats())
```

```
{'feature_count in: mut': 316, 'feature_count in: cna': 1237, 'sample_count': 556}
{'feature_count in: mut': 316, 'feature_count in: cna': 1237, 'sample_count': 238}
```

1.2 Explore sample annotations

- What are the available clinical variables? Are they available in both train and test datasets? (See .ann)

```
In [14]: print("Train Clinical Variables:\n", train_dataset.ann.keys())
          print("Test Clinical Variables:\n", test_dataset.ann.keys())
```

```
Train Clinical Variables:
dict_keys(['AGE', 'OS_MONTHS', 'OS_STATUS', 'KARNOFSKY_PERFORMANCE_SCORE', 'STUDY', 'BCR_STATUS', 'HISTOLOGICAL_DIAGNOSIS', 'SEX'])
Test Clinical Variables:
dict_keys(['AGE', 'OS_MONTHS', 'OS_STATUS', 'KARNOFSKY_PERFORMANCE_SCORE', 'STUDY', 'BCR_STATUS', 'HISTOLOGICAL_DIAGNOSIS', 'SEX'])
```

```
In [15]: train_clinical_vars = set(train_dataset.ann.keys())
          test_clinical_vars = set(test_dataset.ann.keys())

          common_vars = train_clinical_vars.intersection(test_clinical_vars)

          print("✅ Available in both datasets: ", common_vars)
```

```
✅ Available in both datasets: {'OS_MONTHS', 'SEX', 'STUDY', 'HISTOLOGICAL_DIAGNOSIS', 'KARNOFSKY_PERFORMANCE_SCORE', 'OS_STATUS', 'AGE', 'BCR_STATUS'}
```

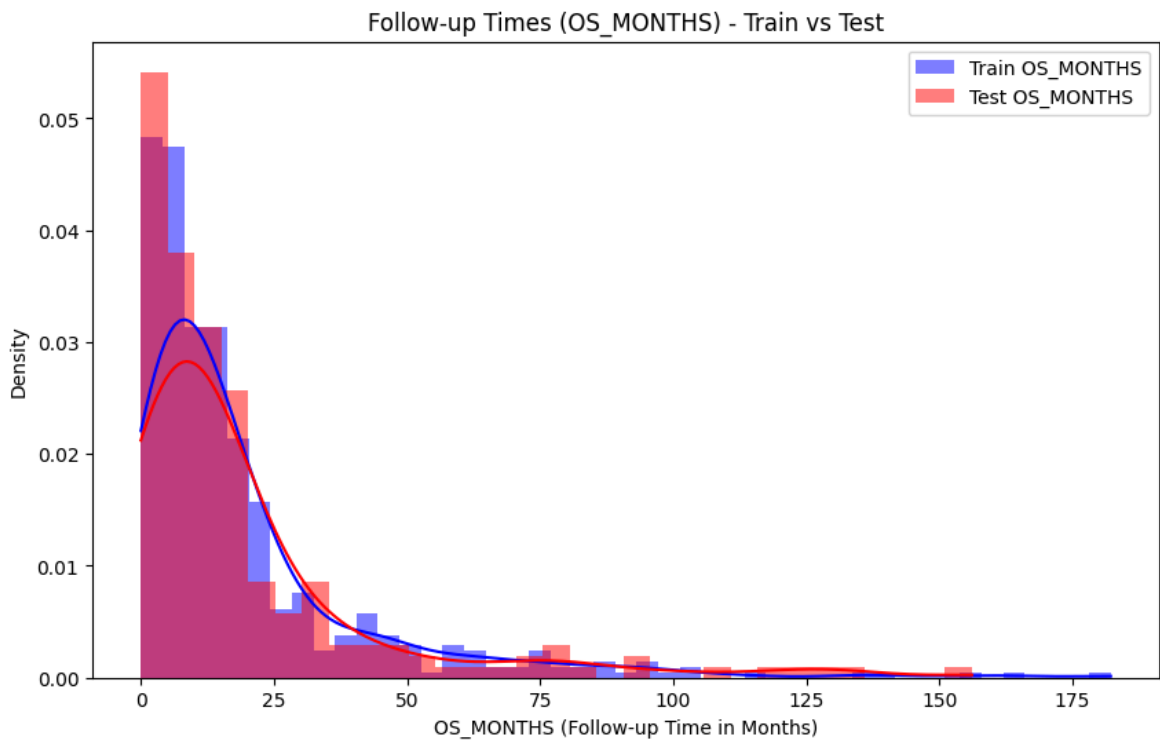
- Make a histogram plot of the follow up times in months (OS_MONTHS) (use sns.histplot)

```
In [18]: import seaborn as sns
import matplotlib.pyplot as plt

# For histogram :
plt.figure(figsize=(10, 6))

sns.histplot(train_dataset.ann['OS_MONTHS'], label='Train OS_MONTHS', color='blue', kde=True)
sns.histplot(test_dataset.ann['OS_MONTHS'], label='Test OS_MONTHS', color='red', kde=True)

plt.legend()
plt.title("Follow-up Times (OS_MONTHS) - Train vs Test")
plt.xlabel("OS_MONTHS (Follow-up Time in Months)")
plt.ylabel("Density")
plt.show()
```

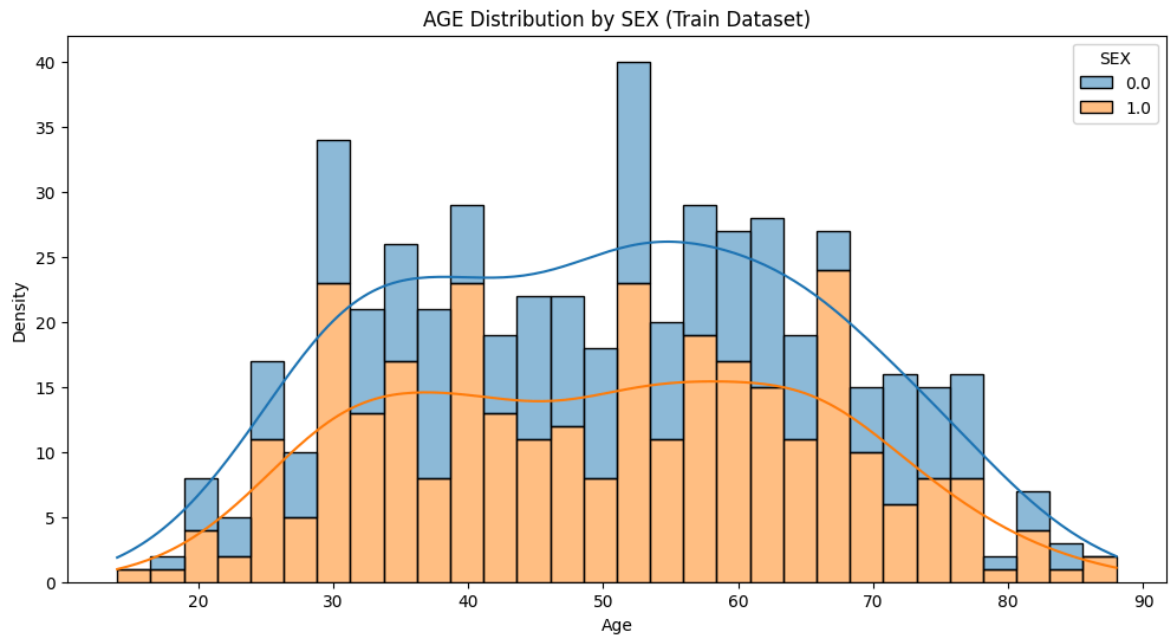


- Make a histogram of the age distribution of the patients in the training data; facet the histogram by "SEX" variable (see flexynesis.utils.plot_boxplot)

```
In [19]: plt.figure(figsize=(12, 6))

sns.histplot(data=train_dataset.ann, x="AGE", hue="SEX", kde=True, multiple=True)

plt.title("AGE Distribution by SEX (Train Dataset)")
plt.xlabel("Age")
plt.ylabel("Density")
plt.show()
```



In [21]: `help(plot_boxplot)`

Help on function plot_boxplot in module flexynesis.utils:

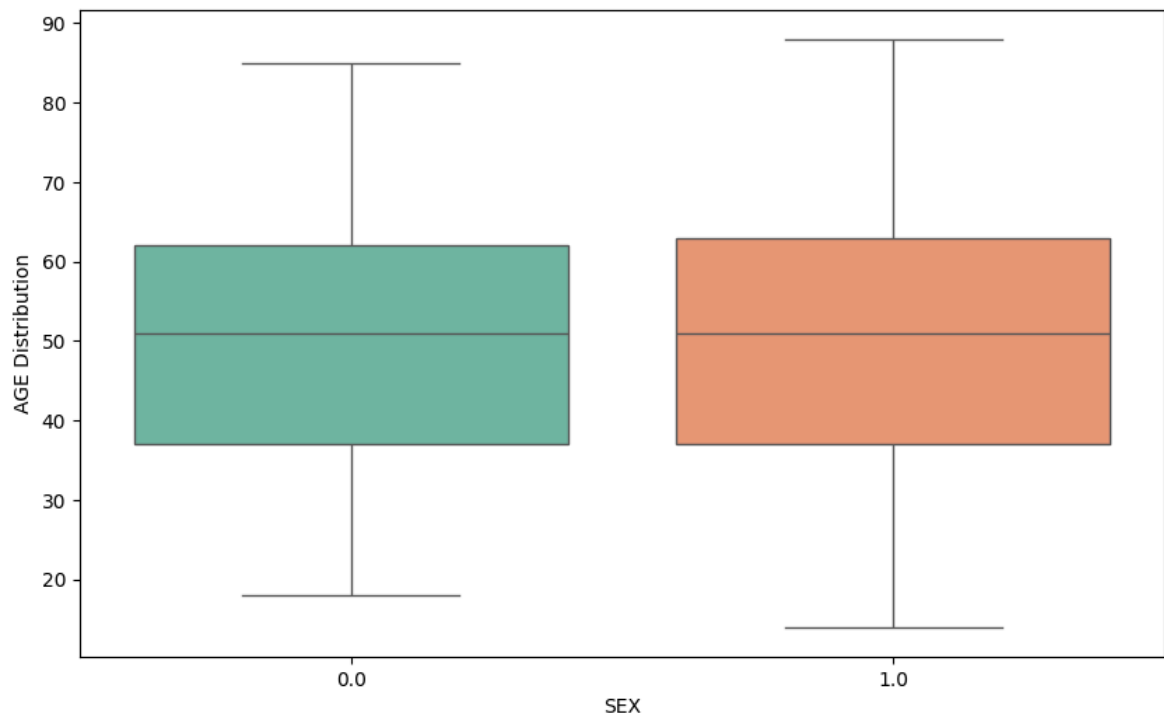
`plot_boxplot(categorical_x, numerical_y, title_x='Categories', title_y='Values')`

In [22]: `from flexynesis.utils import plot_boxplot`

`plot_boxplot(train_dataset.ann['SEX'], train_dataset.ann['AGE'], title_x=`

`/usr/local/lib/python3.11/site-packages/flexynesis/utils.py:155: FutureWarning:`

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.



- Make a summary of all available clinical variables (see `flexynesis.print_summary_stats`)

```
In [34]: help(print_summary_stats)
```

Help on function print_summary_stats in module flexynesis.utils:

```
print_summary_stats(dataset)
    # Accepts as input a MultiOmicDataset object and prints summary stats
    per variable
```

```
In [40]: print_summary_stats(train_dataset)
```

```

Summary for variable: AGE
Numerical Variable Summary: Median = 51.0, Mean = 50.33781190019194
-----
Summary for variable: OS_MONTHS
Numerical Variable Summary: Median = 11.6, Mean = 19.090978886756233
-----
Summary for variable: OS_STATUS
Numerical Variable Summary: Median = 0.0, Mean = 0.36153846153846153
-----
Summary for variable: KARNOFSKY_PERFORMANCE_SCORE
Numerical Variable Summary: Median = 80.0, Mean = 82.45454545454545
-----
Summary for variable: STUDY
Categorical Variable Summary:
  Label: Brain Lower Grade Glioma, Count: 353
  Label: Glioblastoma multiforme, Count: 203
-----
Summary for variable: BCR_STATUS
Categorical Variable Summary:
  Label: IGC, Count: 454
  Label: NCH, Count: 102
-----
Summary for variable: HISTOLOGICAL_DIAGNOSIS
Categorical Variable Summary:
  Label: astrocytoma, Count: 115
  Label: glioblastoma, Count: 201
  Label: oligoastrocytoma, Count: 79
  Label: oligodendroglioma, Count: 126
  Label: nan, Count: 35
-----
Summary for variable: SEX
Categorical Variable Summary:
  Label: Female, Count: 209
  Label: Male, Count: 312
  Label: nan, Count: 35
-----

```

- Notice that the categorical variables such as "SEX", "STUDY", "HISTOLOGICAL_DIAGNOSIS" are encoded numerically in the "dataset.ann" objects. Use dataset.label_mappings to map the STUDY variable to their original labels. Print the top 10 values in dataset.ann['STUDY'] and the mapped label values.

```

In [41]: study_values = train_dataset.ann['STUDY']
         study_labels = train_dataset.label_mappings['STUDY']

         print("Top 10 Values in 'STUDY' (Encoded):")
         print(study_values[:10])

```

```

Top 10 Values in 'STUDY' (Encoded):
tensor([0., 1., 0., 0., 0., 0., 0., 0., 1., 0.], dtype=torch.float64)

```

```

In [44]: print("\nTop 10 Mapped Labels in 'STUDY':")
         mapped_labels = [study_labels[int(val)]
                        for val in study_values[:10]]
         print(mapped_labels)

```

Top 10 Mapped Labels in 'STUDY':

```
['Brain Lower Grade Glioma', 'Glioblastoma multiforme', 'Brain Lower Grade Glioma', 'Brain Lower Grade Glioma', 'Brain Lower Grade Glioma', 'Brain Lower Grade Glioma', 'Brain Lower Grade Glioma', 'Brain Lower Grade Glioma', 'Glioblastoma multiforme', 'Brain Lower Grade Glioma']
```

- Now, let's explore the data matrices. Make a PCA plot of the mutation data matrix and color the samples by "HISTOLOGICAL_DIAGNOSIS". See `flexynesis.plot_dim_reduced` function

First create a pandas data frame with the data matrix of interest with feature and sample names

```
df = pd.DataFrame(train_dataset.dat['cna'], index =
train_dataset.samples, columns= train_dataset.features['cna'])
```

Check the data frame contents

```
df.head()
```

Make a PCA plot of CNA values using the labels from the STUDY variable

Note: if you couldn't map the labels above, you can also use `train_dataset.dat['STUDY']` as labels

```
In [45]: import pandas as pd

# CNA -> DataFrame
df_cna = pd.DataFrame(train_dataset.dat['cna'], index=train_dataset.samples, columns=train_dataset.features['cna'])

print(df_cna.head())
```

	SLC30A8	ZNF273	OR9A1P	AGL	KCNA5	MIR603
TCGA-HT-8113	-0.254134	-0.747855	-0.871568	-0.072414	-0.207080	0.542008
TCGA-14-0871	0.803414	1.973204	1.884189	2.942561	-0.499669	-1.218370
TCGA-HT-A618	-0.190997	-0.699527	-2.050016	0.470825	-0.148562	0.593586
TCGA-DU-A7TD	-0.243611	2.271574	2.385029	0.389339	-0.201760	-1.126427
TCGA-HT-7485	-0.222565	-0.733146	-0.855704	0.383907	-0.241659	0.577889

	SNTB1	MRPL13	MTBP	SNORA72	ENSG00000252158.1	...
TCGA-HT-8113	-0.263039	-0.262880	-0.262880		-0.265474	...
TCGA-14-0871	0.811422	0.811559	0.811559		0.803367	...
TCGA-HT-A618	-0.086635	-0.172007	-0.172007		-0.108605	...
TCGA-DU-A7TD	-0.252348	-0.252189	-0.252189		-0.254839	...
TCGA-HT-7485	-0.230965	-0.230807	-0.230807		-0.233569	...

	CAV1	FZD1	BCAP29	MNX1	ADAM22	LRP8
TCGA-HT-8113	-0.875647	-0.821836	-0.876168	-0.852175	-0.797862	-0.055958
TCGA-14-0871	1.880812	1.879839	1.903375	1.873636	1.760577	2.959935
TCGA-HT-A618	-0.823510	-0.770735	-0.823594	-2.017817	-0.749471	0.487446
TCGA-DU-A7TD	2.381781	2.370850	2.408539	2.274886	2.225557	0.405936
TCGA-HT-7485	-0.859779	-0.806283	-0.860167	-0.836483	-0.783135	0.400502

	NOM1	RN7SL290P	CEP41	snoU13	ENSG00000239044.1
TCGA-HT-8113	-0.852356	-0.050895	-0.892950		-0.892950
TCGA-14-0871	1.873364	2.995690	1.817840		1.817840
TCGA-HT-A618	-2.017960	0.498039	-2.052170		-2.052170
TCGA-DU-A7TD	2.274601	0.415699	2.310508		2.310508
TCGA-HT-7485	-0.836665	0.410210	-0.877346		-0.877346

[5 rows x 1237 columns]

In [47]: `help(plot_dim_reduced)`

Help on function plot_dim_reduced in module flexynesis.utils:

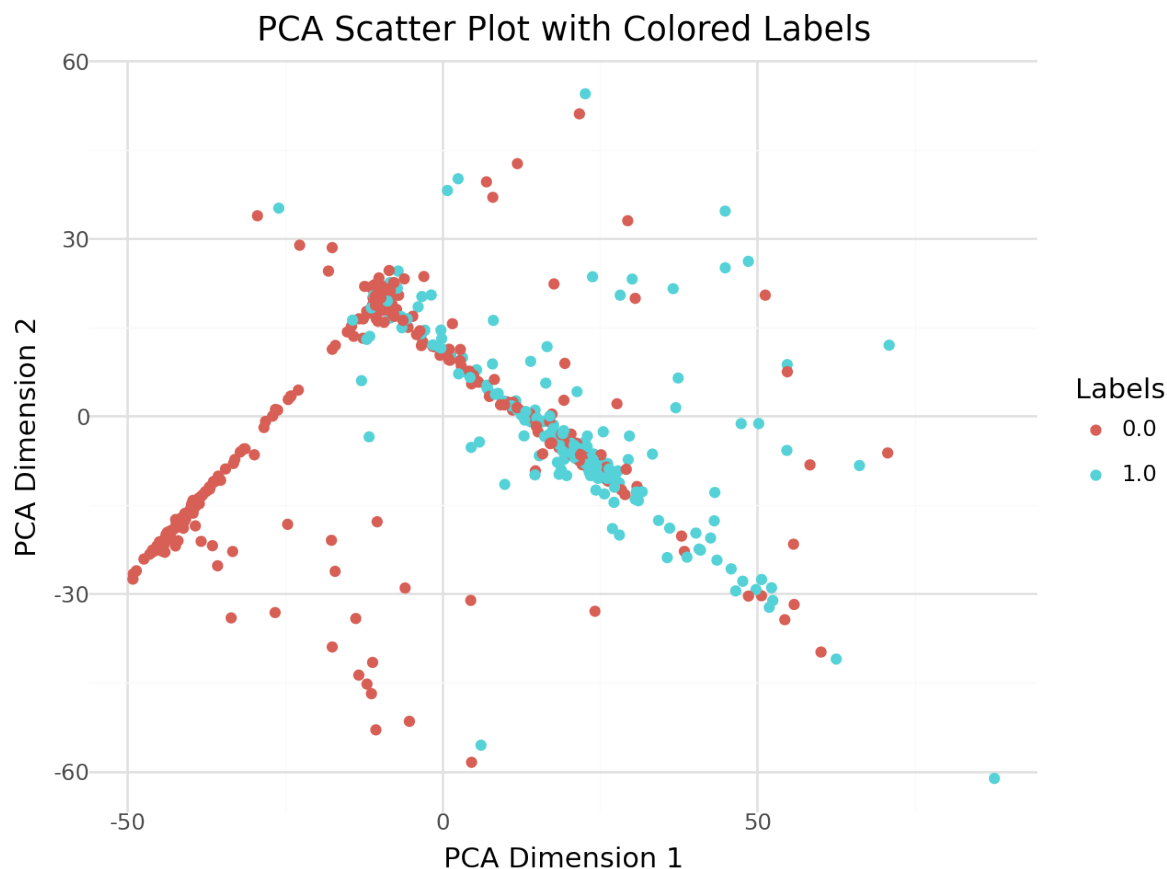
`plot_dim_reduced(matrix, labels, method='pca', color_type='categorical')`
 Plots the first two dimensions of the transformed input matrix using `plotnine` (ggplot2-like API),
 with points colored based on the provided labels. The transformation method can be either PCA or UMAP.

Args:

`matrix` (np.array): Input data matrix (n_samples, n_features).
`labels` (list): List of labels (strings or integers).
`method` (str): Transformation method ('pca' or 'umap'). Default is 'pca'.
`color_type` (str): Type of the color scale ('categorical' or 'numerical'). Default is 'categorical'.

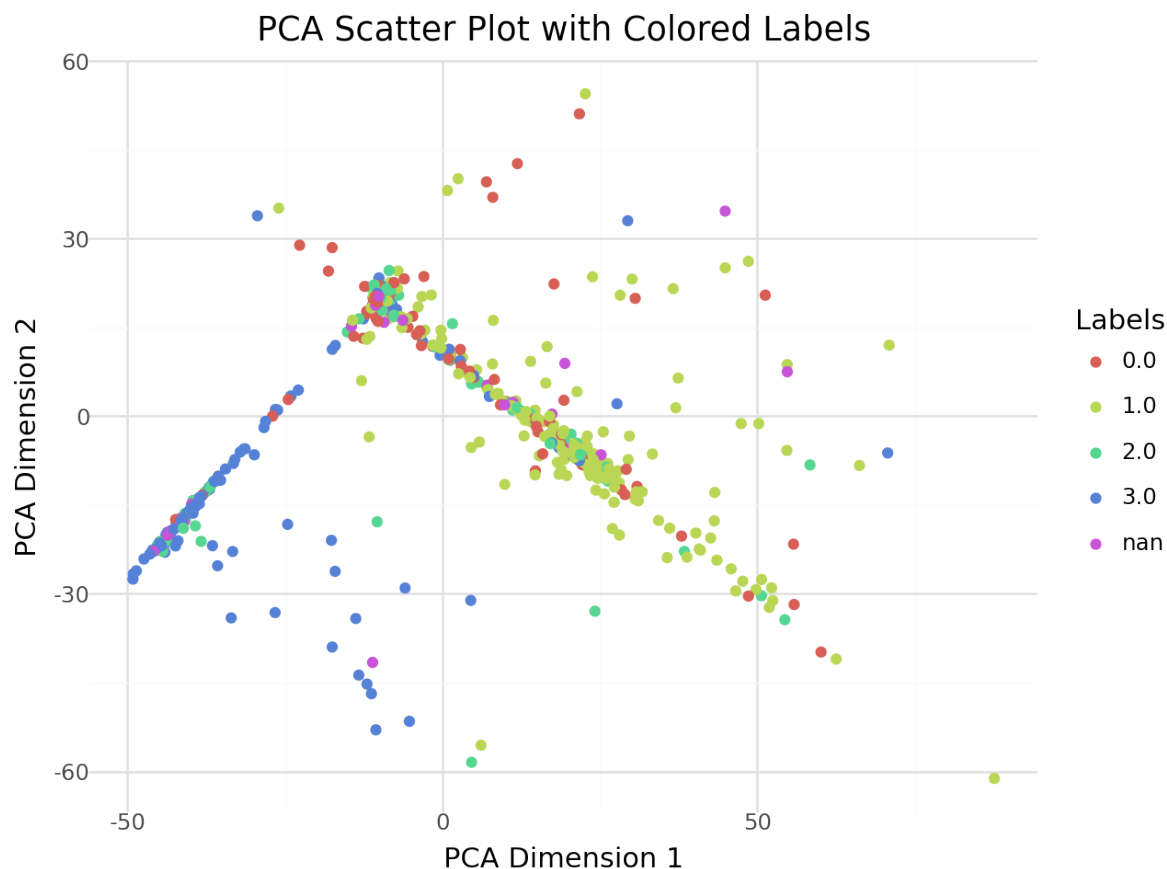
In [49]: `from flexynesis import plot_dim_reduced`

`plot_dim_reduced(df_cna.values, labels=train_dataset.ann['STUDY'], method`

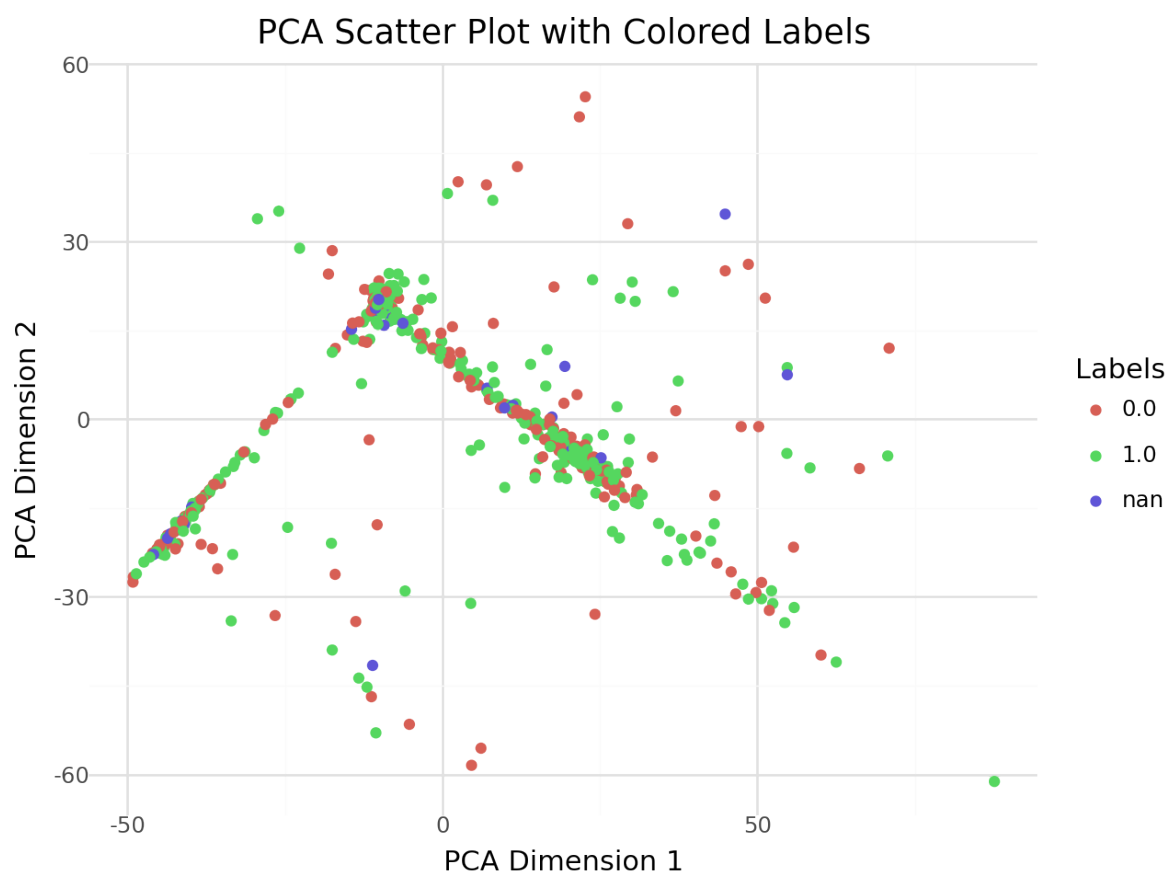


- (Optional exercise ideas):
 - Make a PCA plot coloring the samples by HISTOLOGICAL_DIAGNOSIS, GENDER, or any other clinical variable
 - Repeat the same exercise on the mutation data matrix.

```
In [50]: # HISTOLOGICAL_DIAGNOSIS
plot_dim_reduced(df_cna.values, labels=train_dataset.ann['HISTOLOGICAL_DI
               method='pca', color_type='categorical')
```

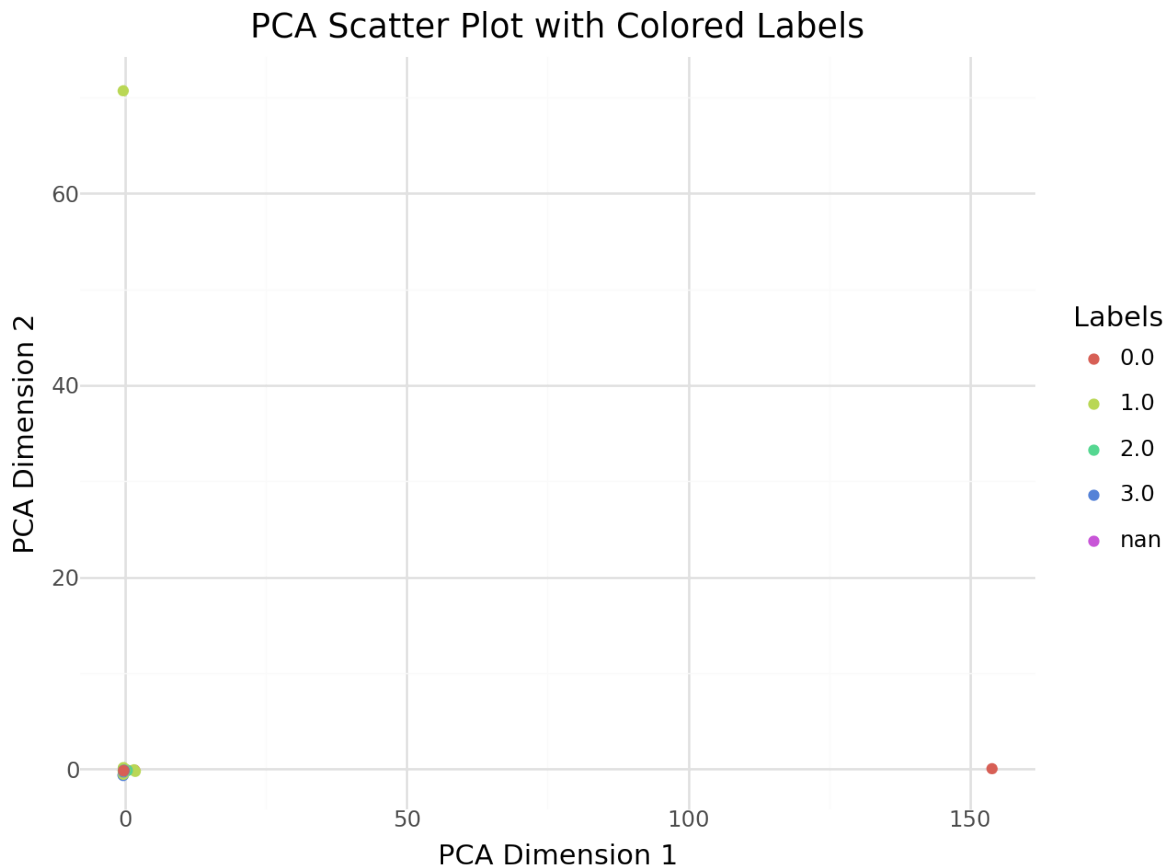


```
In [51]: # GENDER
plot_dim_reduced(df_cna.values, labels=train_dataset.ann['SEX'], method='')
```



```
In [52]: # DataFrame for mutations dataset
df_mut = pd.DataFrame(train_dataset.dat['mut'],
                      index=train_dataset.samples, columns=train_dataset.
```

```
# in mutation dataset (HISTOLOGICAL_DIAGNOSIS)
plot_dim_reduced(df_mut.values, labels=train_dataset.ann['HISTOLOGICAL_DI
```



2. Training a single model using manually set hyperparameters

Now that we have familiarized ourselves with the dataset at hand, we can start building models.

First we will do a single model training by manually setting hyperparameters. Based on the model performance, we will try modifying individual hyperparameters and build more and more models and see if we can improve model performance.

We will need to define the following components for starting a model training:

1. Split the train_dataset into train/validation components
2. Define data loaders for both train and validation splits
3. Define a pytorch-lightning trainer
4. Define a model with hyperparameters
5. Fit the model

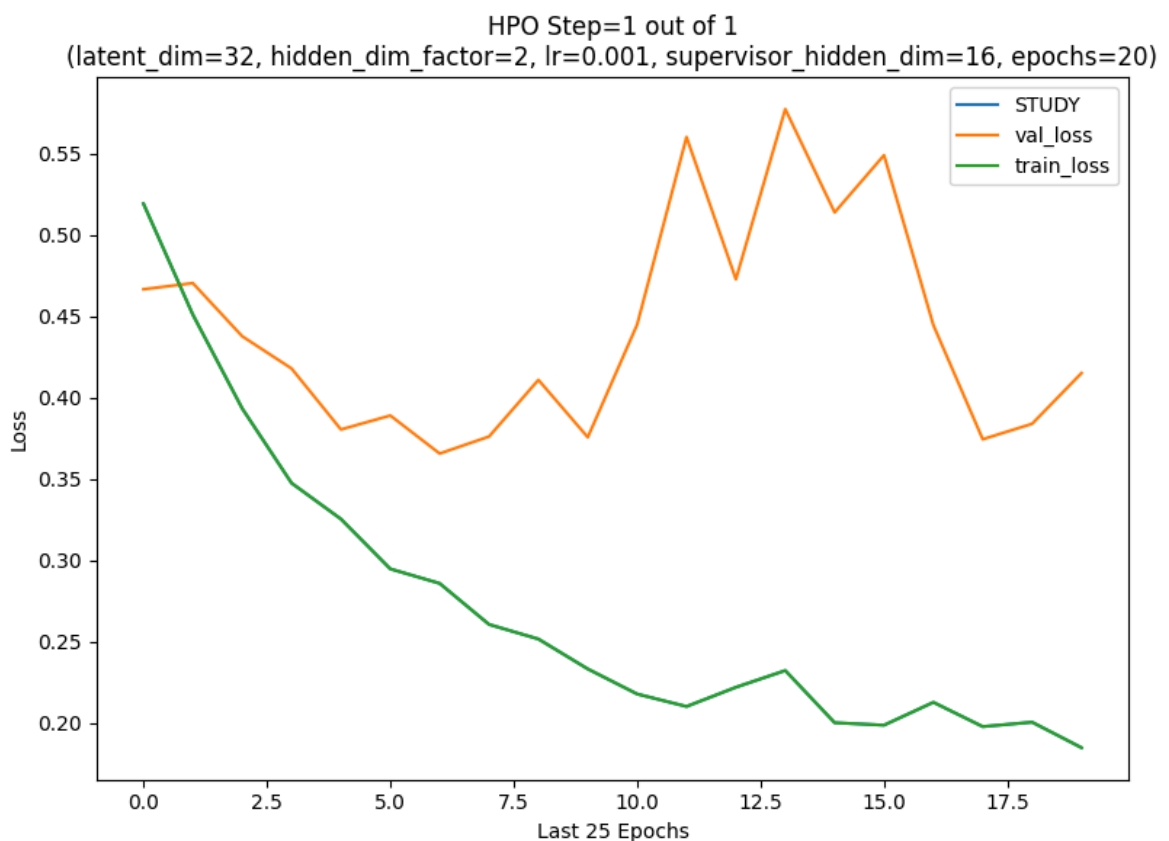
```
In [54]: # randomly assign 80% of samples for training, 20% for validation
train_indices = random.sample(range(0, len(train_dataset)), int(len(train_dataset)*0.8))
val_indices = list(set(range(len(train_dataset))) - set(train_indices))
train_subset = train_dataset.subset(train_indices)
val_subset = train_dataset.subset(val_indices)
```

```
# define data loaders for train/validation splits
from torch.utils.data import DataLoader
train_loader = DataLoader(train_subset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_subset, batch_size=32, shuffle=False)
```

Now, we need to define a model with manually set hyperparameters and a lightning-trainer fit the model.

Notice: Notice the callback we are passing to the trainer which enables us to plot the loss values as the training progresses.

```
In [55]: # Define a model with manually set hyperparameters for the DirectPred model
myparams = {'latent_dim': 32, 'hidden_dim_factor': 2, 'lr': 0.001, 'supervisor_hidden_dim': 16, 'epochs': 20}
model = flexynesis.DirectPred(config = myparams, dataset = train_dataset,
trainer = pl.Trainer(max_epochs=myparams['epochs'], default_root_dir=".",
callbacks=[flexynesis.LiveLossPlot(myparams, 1, 1)])
# Fit the model
trainer.fit(model, train_loader, val_loader)
```



``Trainer.fit` stopped: `max_epochs=20` reached.`

While we can observe how well the model training went based on the "loss" values, we can also evaluate the model performance on test dataset

```
In [56]: # evaluate the model performance on predicting the target variable
flexynesis.evaluate_wrapper("DirectPred", model.predict(test_dataset), te
```

Out [56]:

	method	var	variable_type	metric	value
0	DirectPred	STUDY	categorical	balanced_acc	0.796732
1	DirectPred	STUDY	categorical	f1_score	0.804493
2	DirectPred	STUDY	categorical	kappa	0.579821
3	DirectPred	STUDY	categorical	average_auroc	0.883045
4	DirectPred	STUDY	categorical	average_aupr	0.780892

2.1 Exercise

- Now, repeat the above model training and evaluation by manually changing the hyperparameters (Try at least 5 different combinations)
- See if you can find a better hyperparameter combination that yields a better classification performance than the initial setup we provided.
- See the default hyperparameter ranges we use for Flexynesis here: <https://github.com/BIMSBbioinfo/flexynesis/blob/69b92ca9370551e9fcc82a756cb4/> but feel free to try outside these ranges too.
- Also try to observe the impact of the changing parameters on how the train/validation loss curves change.

```

myparams = {'latent_dim': XX, 'hidden_dim_factor': XX,
            'lr': XX, 'supervisor_hidden_dim': XX, 'epochs': XX}

model = flexynesis.DirectPred(config = myparams, dataset =
train_dataset, target_variables=['STUDY'])

trainer = pl.Trainer(max_epochs=myparams['epochs'],
default_root_dir=".", logger=False,
enable_checkpointing=False,
                    callbacks=
[flexynesis.LiveLossPlot(myparams, 1, 1)])

trainer.fit(model, train_loader, val_loader)

flexynesis.evaluate_wrapper("DirectPred",
model.predict(test_dataset), test_dataset)

```

```

In [57]: myparams = {
            'latent_dim': 32,
            'hidden_dim_factor': 0.4,
            'lr': 0.001,
            'supervisor_hidden_dim': 16,
            'epochs': 20
        }

```

```

In [58]: # Dataset ->> 80% train, 20% val : from [54]

train_indices = random.sample(range(0, len(train_dataset)), int(len(train
val_indices = list(set(range(len(train_dataset)) - set(train_indices))

```

```
train_subset = train_dataset.subset(train_indices)
val_subset = train_dataset.subset(val_indices)
```

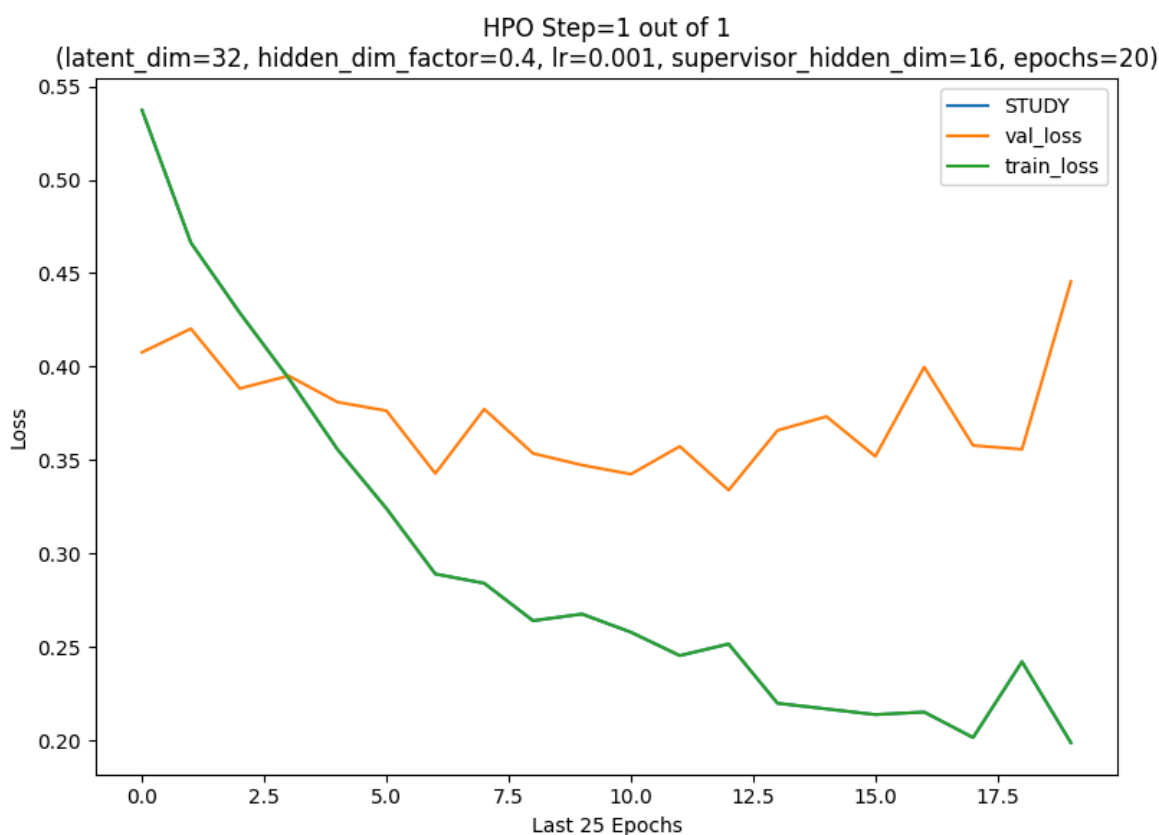
```
In [59]: train_loader = DataLoader(train_subset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_subset, batch_size=32, shuffle=False)
```

```
In [60]: model = flexynesis.DirectPred(config=myparams, dataset=train_dataset, tar
```

```
In [61]: trainer = pl.Trainer(max_epochs=myparams['epochs'], default_root_dir="./"
        callbacks=[flexynesis.LiveLossPlot(myparams, 1, 1)])
```

```
GPU available: False, used: False
TPU available: False, using: 0 TPU cores
HPU available: False, using: 0 HPUs
```

```
In [62]: trainer.fit(model, train_loader, val_loader)
```



```
`Trainer.fit` stopped: `max_epochs=20` reached.
```

```
In [63]: flexynesis.evaluate_wrapper("DirectPred", model.predict(test_dataset), te
```

```
Out[63]:
```

	method	var	variable_type	metric	value
0	DirectPred	STUDY	categorical	balanced_acc	0.749020
1	DirectPred	STUDY	categorical	f1_score	0.778142
2	DirectPred	STUDY	categorical	kappa	0.511409
3	DirectPred	STUDY	categorical	average_auroc	0.848597
4	DirectPred	STUDY	categorical	average_aupr	0.727124

Warning!!: In reality, we don't select the best models based on performance on the test dataset.

The best model is selected based on the validation loss value, where the model parameters that yields the lowest validation loss is selected to be the best model.

The validation dataset which we use to compute the validation loss is basically a subset of the training dataset.

3. Automating the Hyperparameter Optimisation Procedure

What we did in the above section was to set random hyperparameters, build a model, evaluate the model and try different hyperparameters based on our previous model performance. However, this process can be quite time consuming and arbitrary. This process can be automated using a Bayesian approach, where the model training is sequentially done for a number of hyperparameter optimisation iterations.

Now, we are ready to do a model training using hyperparameter optimisation.

- `model_class` : We pick `DirectPred` (a fully connected network) for now.
- `config_name` : We use the default/built-in hyperparameter search space for `DirectPred` class.
- `target_variables` : 'STUDY' variable contains the type of disease
- `n_iter` : We do 5 iterations of hyperparameter optimisation. For demonstration purposes, we set it to a small number.
- `plot_losses` : We want to visualize how the training progresses.
- `early_stop_patience` : If a training does not show any signs of improving the performance on the `validation` part of the `train_dataset` for at least 10 epochs, we stop the training. This not only significantly decreases the amount spent on training by avoiding unnecessary continuation of unpromising training runs, but also helps avoid over-fitting the network on the training data.

Note 1: Notice how the hyperparameters using in different HPO steps change at each iteration.

Note 2: Also notice that we are running the model for more epochs (500 by default) however, by using "early_stop_patience=10", we avoid lengthy training when validation performance is not improving.

Note 3: Try to follow the the loss curves and the used hyperparameters. See if you can spot which combination yields the lowest/best loss values.

Warning!!: In reality we need to set `n_iter` to higher values so that the optimizer can collect enough data points to learn trends in the parameter space.

```
In [64]: # Define a tuner; See n_iter is the number of
tuner = flexsynthesis.HyperparameterTuning(train_dataset,
```

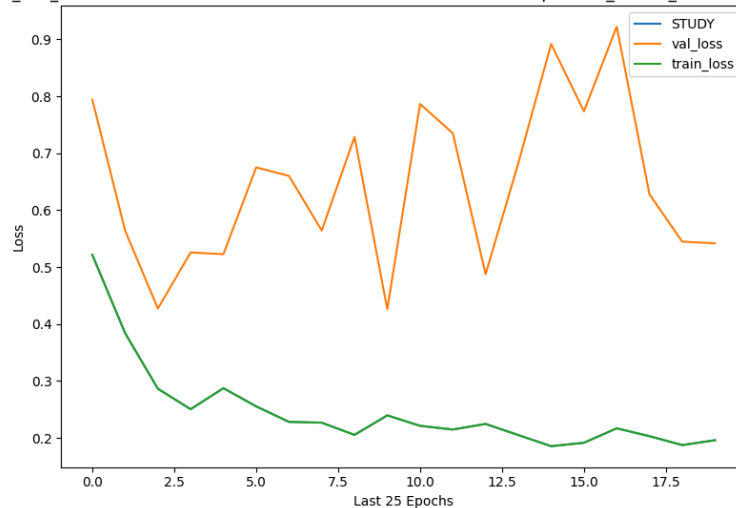
```

model_class = flexynesis.DirectPr
config_name = "DirectPred",
target_variables = ['STUDY'],
n_iter=5,
plot_losses=True,
early_stop_patience=10)

### Perform Training
model, best_params = tuner.perform_tuning()

```

HPO Step=5 out of 5
(latent_dim=92, hidden_dim_factor=0.297250998589454, lr=0.005864329311456449, supervisor_hidden_dim=15, epochs=500, batch_size=32)



Validation: |
?it/s]

| 0/? [00:00<?,

Validate metric	DataLoader 0
STUDY	0.5418112277984619
val_loss	0.5418112277984619

Tuning Progress: 100%|█| 5/5 [00:27<00:00, 5.57s/it, Iteration=5, Best Loss=0.3

[INFO] current best val loss: 0.36638888716697693; best params: {'latent_dim': np.int64(61), 'hidden_dim_factor': 0.20376760394807347, 'lr': 0.00037136163492942274, 'supervisor_hidden_dim': np.int64(11), 'epochs': 500, 'batch_size': np.int64(64)} since 1 hpo iterations

In [65]: *## See which hyperparameter combination was the best*
best_params

Out[65]: {'latent_dim': np.int64(61),
'hidden_dim_factor': 0.20376760394807347,
'lr': 0.00037136163492942274,
'supervisor_hidden_dim': np.int64(11),
'epochs': 65,
'batch_size': np.int64(64)}

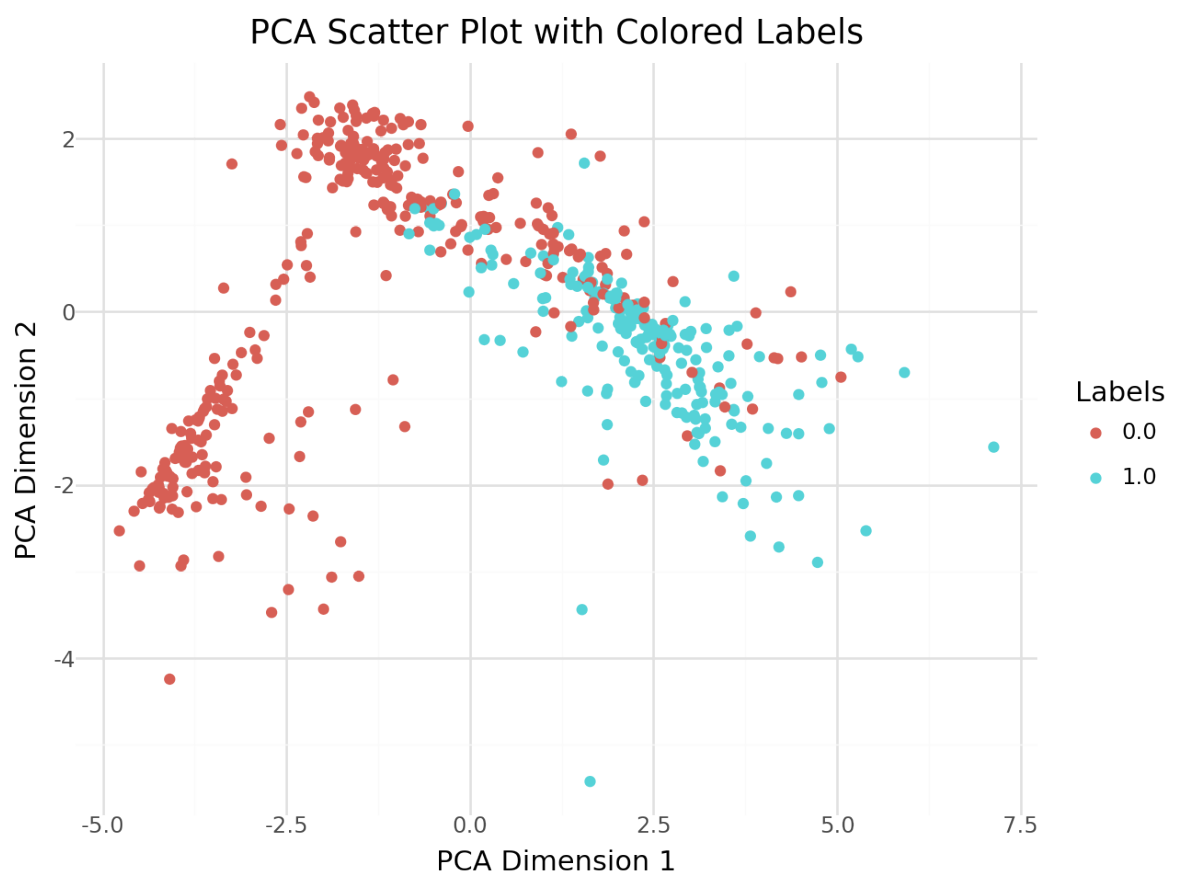
In [66]: *## Evaluate the model and visualising the results*
flexynesis.evaluate_wrapper(method = 'DirectPred', y_pred_dict=model.pred

Out [66]:

	method	var	variable_type	metric	value
0	DirectPred	STUDY	categorical	balanced_acc	0.766013
1	DirectPred	STUDY	categorical	f1_score	0.788740
2	DirectPred	STUDY	categorical	kappa	0.537649
3	DirectPred	STUDY	categorical	average_auroc	0.856671
4	DirectPred	STUDY	categorical	average_aupr	0.706361

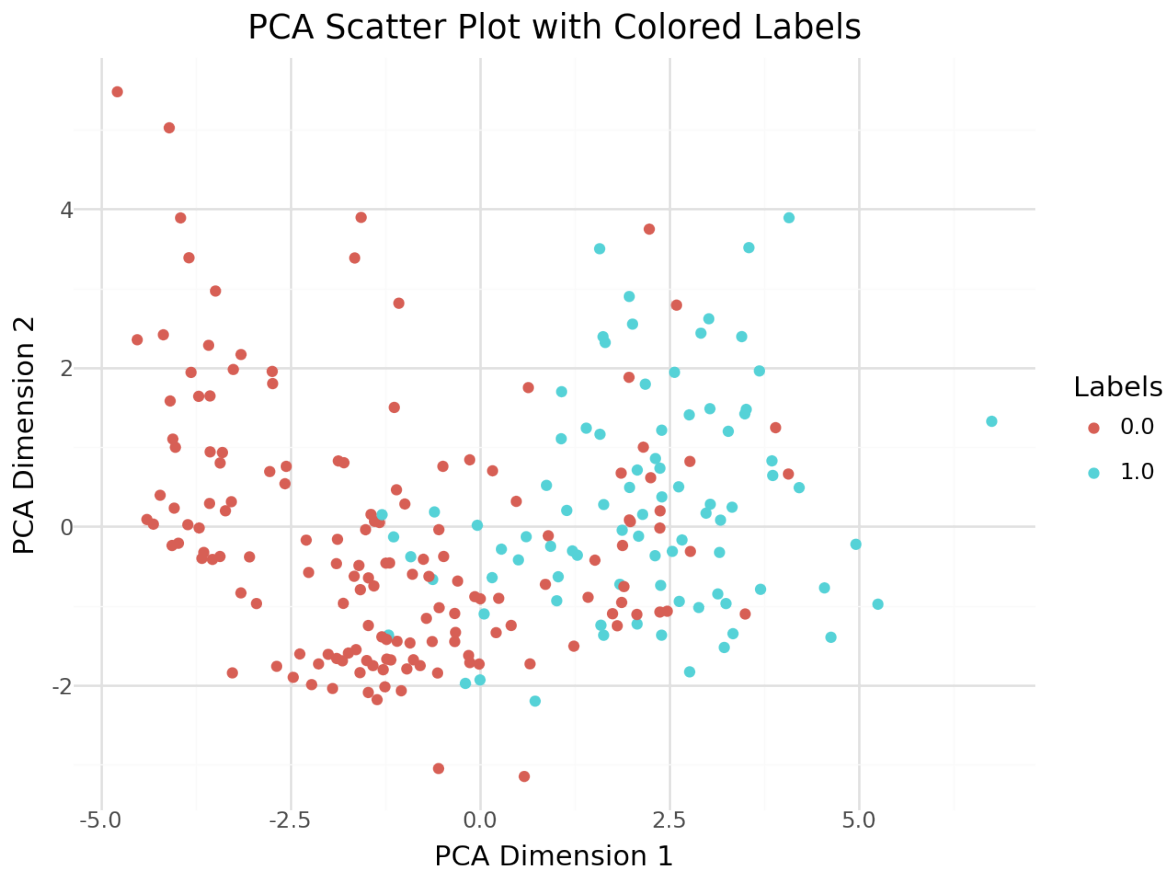
Let's extract the sample embeddings and make a PCA plot and color by the target variable

```
In [67]: train_embeddings = model.transform(train_dataset)
flexynesis.plot_dim_reduced(train_embeddings, train_dataset.ann['STUDY'])
```



Repeat the same for the test dataset: extract sample embeddings for test dataset samples and make a PCA plot, colored by "STUDY" variable

```
In [68]: test_embeddings = model.transform(test_dataset)
flexynesis.plot_dim_reduced(test_embeddings, test_dataset.ann['STUDY'])
```



3.1 Exercises

Exercise 1:

Look up what Harrell's C-index means and write down a simple description of what it measures.

->> ANSWER <<-

Harrell's C-index is a measure used to evaluate the performance of risk models, particularly in predicting outcomes like the time to a certain event. It helps determine how well the model ranks individuals based on their risk scores. The C-index measures whether the model correctly predicts that individuals with higher risk scores experience the event sooner.

The value of the **C-index** ranges from **0 to 1**. A value of 1 means perfect prediction, where the model always ranks people correctly:

- A value of 0.5 indicates random ranking, meaning the model is no better than guessing.
- A value of 0 suggests that the model's predictions are completely reversed, performing worse than a random guess.

Exercise 2:

Now, you build a model using hyperparameter tuning (run at least 10 HPO steps) to predict the survival outcomes of patients. Evaluate the final model on test dataset,

which computes the "C-index".

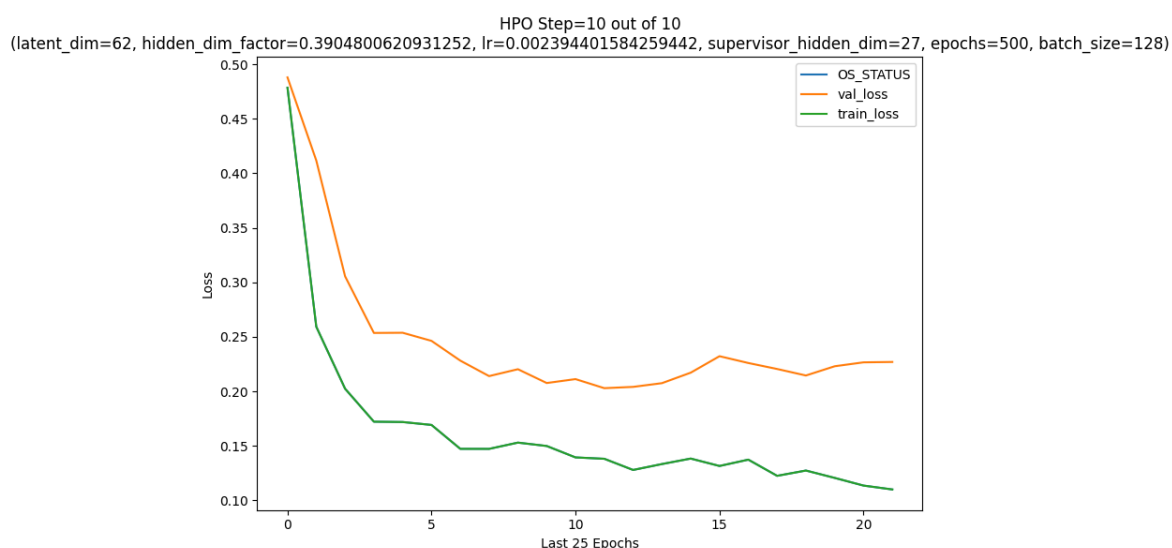
Feel free to cheat from the tutorial available here:

https://github.com/BIMSBbioinfo/flexynesis/blob/main/examples/tutorials/survival_subtyp

See how "OS_STATUS" and "OS_MONTHS" were used.

```
In [69]: tuner = flexynesis.HyperparameterTuning(train_dataset,
                                                model_class = flexynesis.DirectPr
                                                config_name = "DirectPred",
                                                target_variables = ['OS_STATUS'],
                                                n_iter=10,
                                                plot_losses=True,
                                                early_stop_patience=10)
```

```
In [70]: model, best_params = tuner.perform_tuning()
```



Validation: |
?it/s]

| 0/? [00:00<?,

Validate metric	DataLoader 0
OS_STATUS	0.22691230475902557
val_loss	0.22691230475902557

Tuning Progress: 100%|█| 10/10 [01:13<00:00, 7.35s/it, Iteration=10, Best Loss=

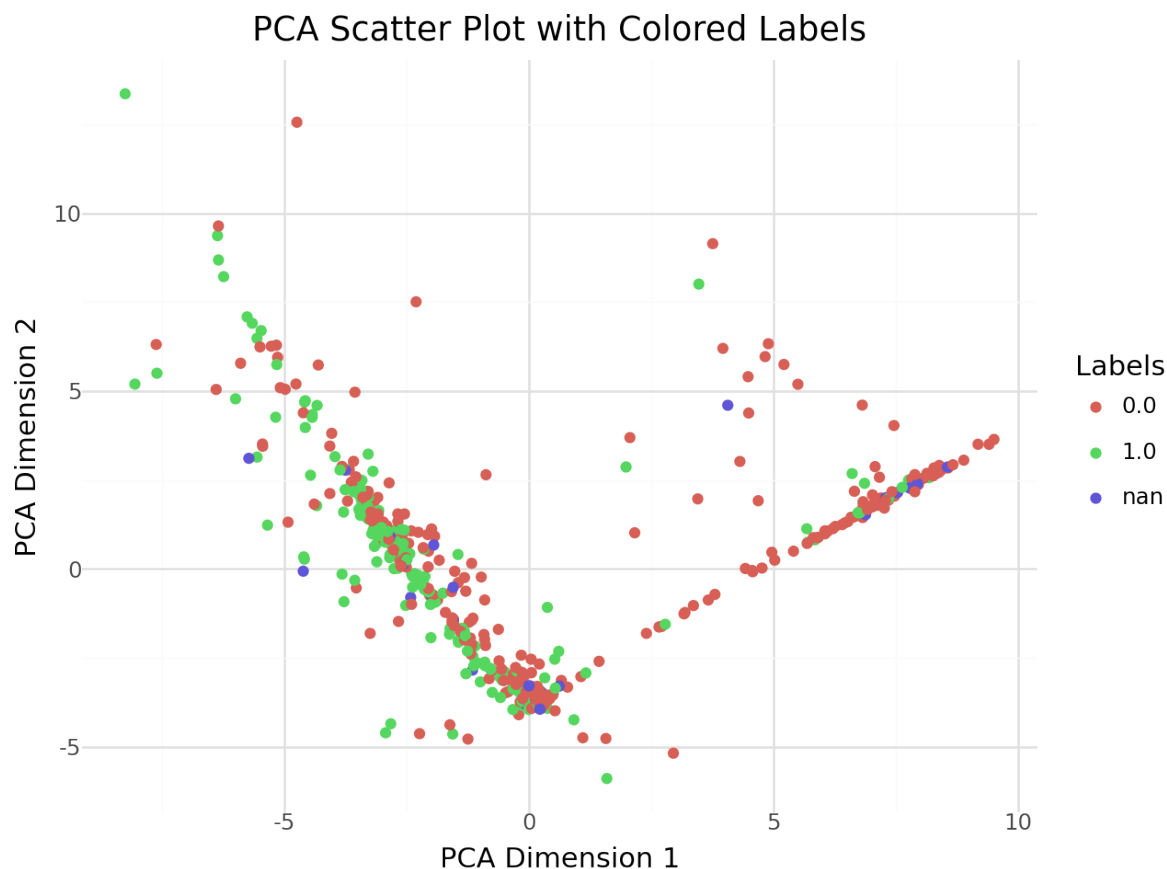
[INFO] current best val loss: 0.19601471722126007; best params: {'latent_dim': np.int64(45), 'hidden_dim_factor': 0.43498634296350847, 'lr': 0.0002542749295012664, 'supervisor_hidden_dim': np.int64(31), 'epochs': 500, 'batch_size': np.int64(32)} since 1 hpo iterations

```
In [71]: flexynesis.evaluate_wrapper(method = 'DirectPred', y_pred_dict=model.pred
```

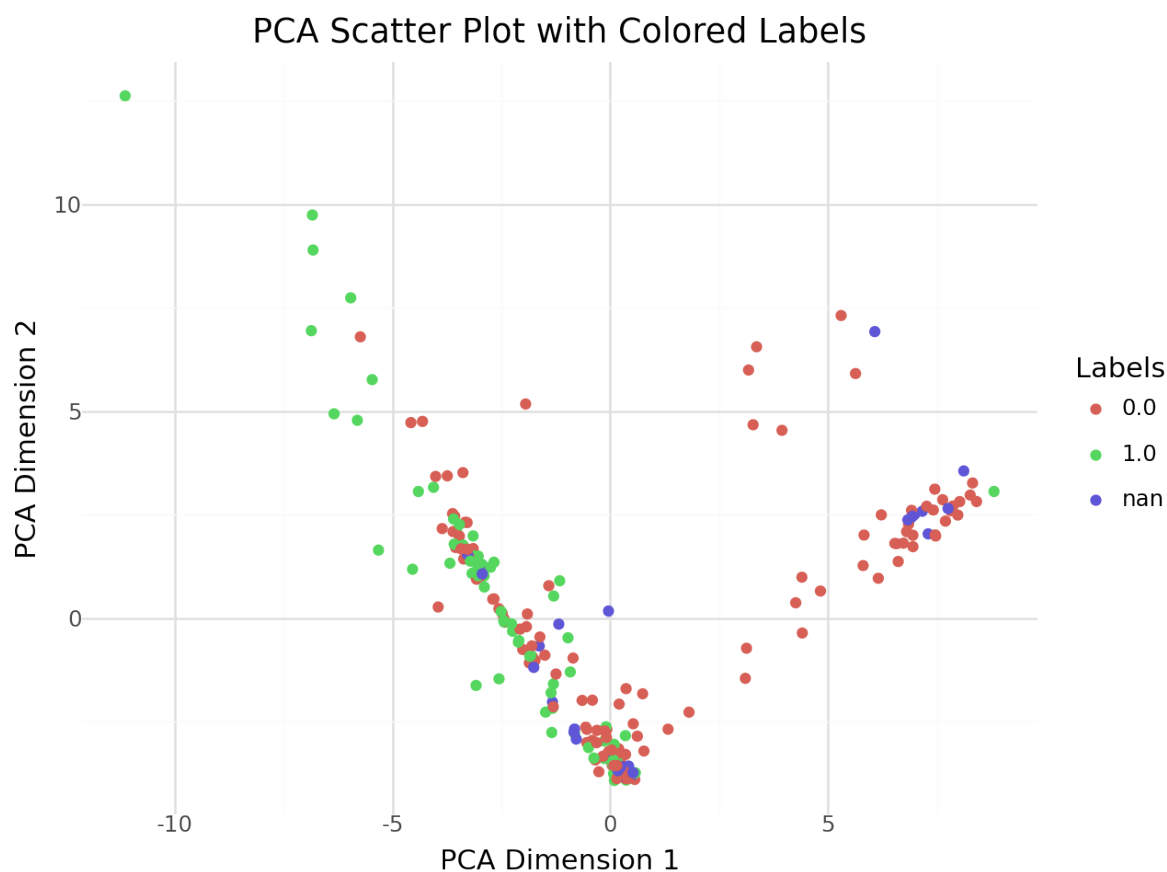
```
Out[71]:
```

	method	var	variable_type	metric	value
0	DirectPred	OS_STATUS	numerical	mse	0.230373
1	DirectPred	OS_STATUS	numerical	r2	0.089111
2	DirectPred	OS_STATUS	numerical	pearson_corr	0.298514

```
In [72]: train_embeddings = model.transform(train_dataset)
flexynesis.plot_dim_reduced(train_embeddings, train_dataset.ann['OS_STATU
```



```
In [73]: test_embeddings = model.transform(test_dataset)
flexynesis.plot_dim_reduced(test_embeddings, test_dataset.ann['OS_STATUS'
```



Exercise 3:

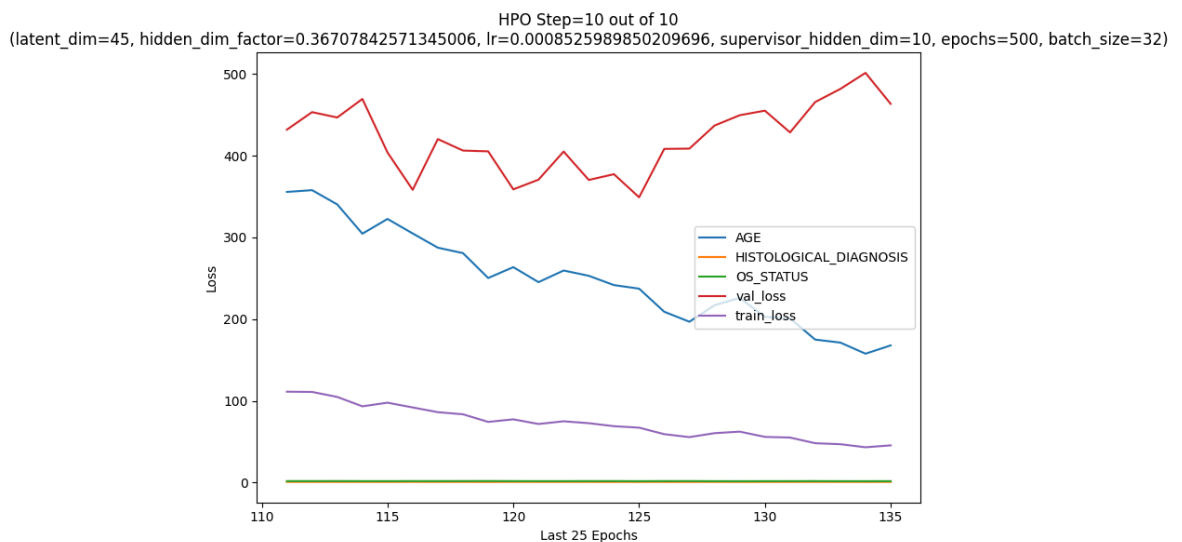
Again build a model using hyperparameter tuning to predict survival outcomes (as in Exercise 1), however, this time use additional clinical variables as targets.

```
flexynesis.HyperparameterTuning(train_dataset,
                                model_class =
flexynesis.DirectPred,
                                config_name =
"DirectPred",
surv_event_var="OS_STATUS",
                                surv_time_var="OS_MONTHS",
                                target_variables = [], =>
What other variables can you use here? Try "AGE" and/or
"HISTOLOGICAL_DIAGNOSIS" and see the model performance
...
```

See if you can get a better C-index using additional target variables.

```
In [74]: tuner = flexynesis.HyperparameterTuning(train_dataset,
                                                    model_class = flexynesis.DirectPr
                                                    config_name = "DirectPred",
                                                    surv_event_var="OS_STATUS",
                                                    surv_time_var="OS_MONTHS",
                                                    target_variables=["AGE", "HISTOLO
                                                    n_iter=10,
                                                    plot_losses=True,
                                                    early_stop_patience=10)

model, best_params = tuner.perform_tuning()
```



Validation: |
?it/s]

| 0/? [00:00<?,

Validate metric	DataLoader 0
AGE HISTOLOGICAL_DIAGNOSIS OS_STATUS val_loss	459.91650390625 1.0187897682189941 2.520674083174083 463.4559677447556

Tuning Progress: 100%|█| 10/10 [07:40<00:00, 46.06s/it, Iteration=10, Best Loss=

[INFO] current best val loss: 173.96353414724538; best params: {'latent_dim': np.int64(79), 'hidden_dim_factor': 0.34887991090541703, 'lr': 0.0005334407949690723, 'supervisor_hidden_dim': np.int64(27), 'epochs': 500, 'batch_size': np.int64(32)} since 5 hpo iterations

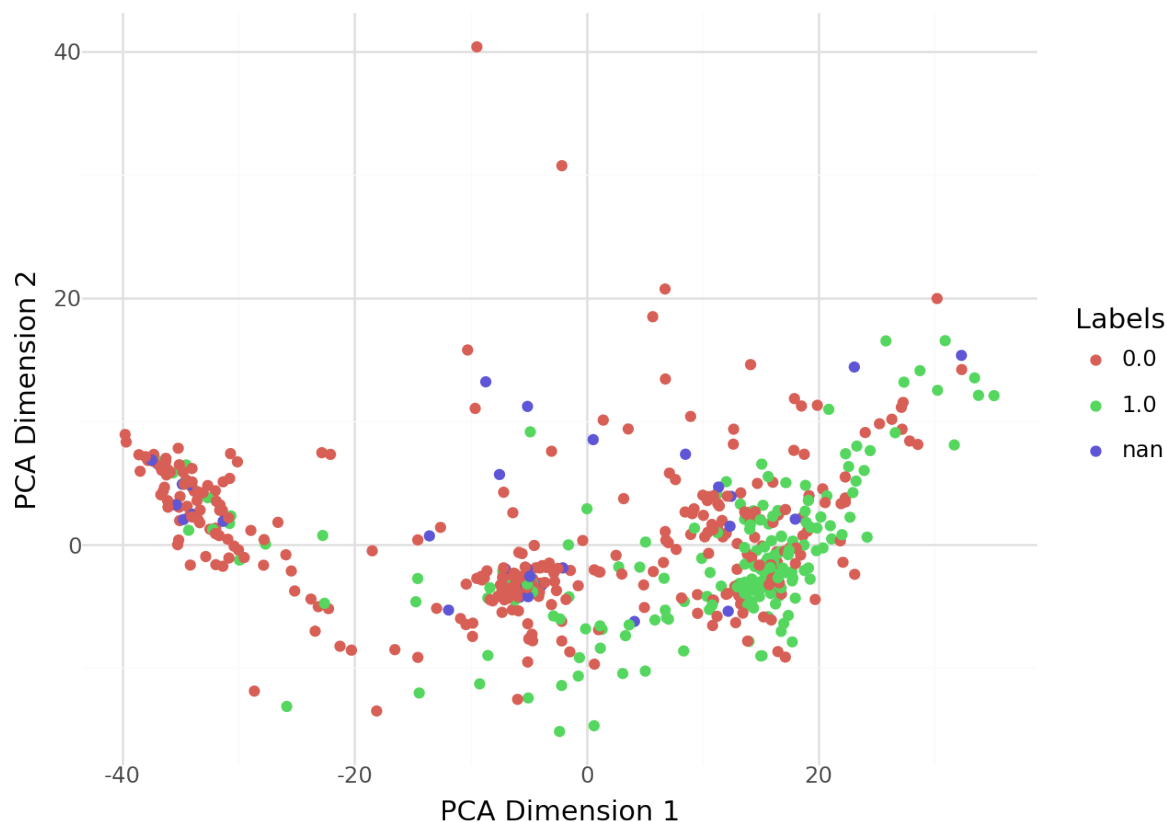
```
In [76]: flexynesis.evaluate_wrapper(method = 'DirectPred',
                                     y_pred_dict=model.predict(test_dataset),
                                     dataset = test_dataset)
```

```
Out [76]:
```

	method	var	variable_type	metric	value
0	DirectPred	AGE	numerical	mse	244.469101
1	DirectPred	AGE	numerical	r2	0.194162
2	DirectPred	AGE	numerical	pearson_corr	0.440638
3	DirectPred	HISTOLOGICAL_DIAGNOSIS	categorical	balanced_acc	0.476920
4	DirectPred	HISTOLOGICAL_DIAGNOSIS	categorical	f1_score	0.515382
5	DirectPred	HISTOLOGICAL_DIAGNOSIS	categorical	kappa	0.334353
6	DirectPred	HISTOLOGICAL_DIAGNOSIS	categorical	average_auroc	NaN
7	DirectPred	HISTOLOGICAL_DIAGNOSIS	categorical	average_aupr	NaN
8	DirectPred	OS_STATUS	numerical	mse	2.148092
9	DirectPred	OS_STATUS	numerical	r2	0.186666
10	DirectPred	OS_STATUS	numerical	pearson_corr	0.432048

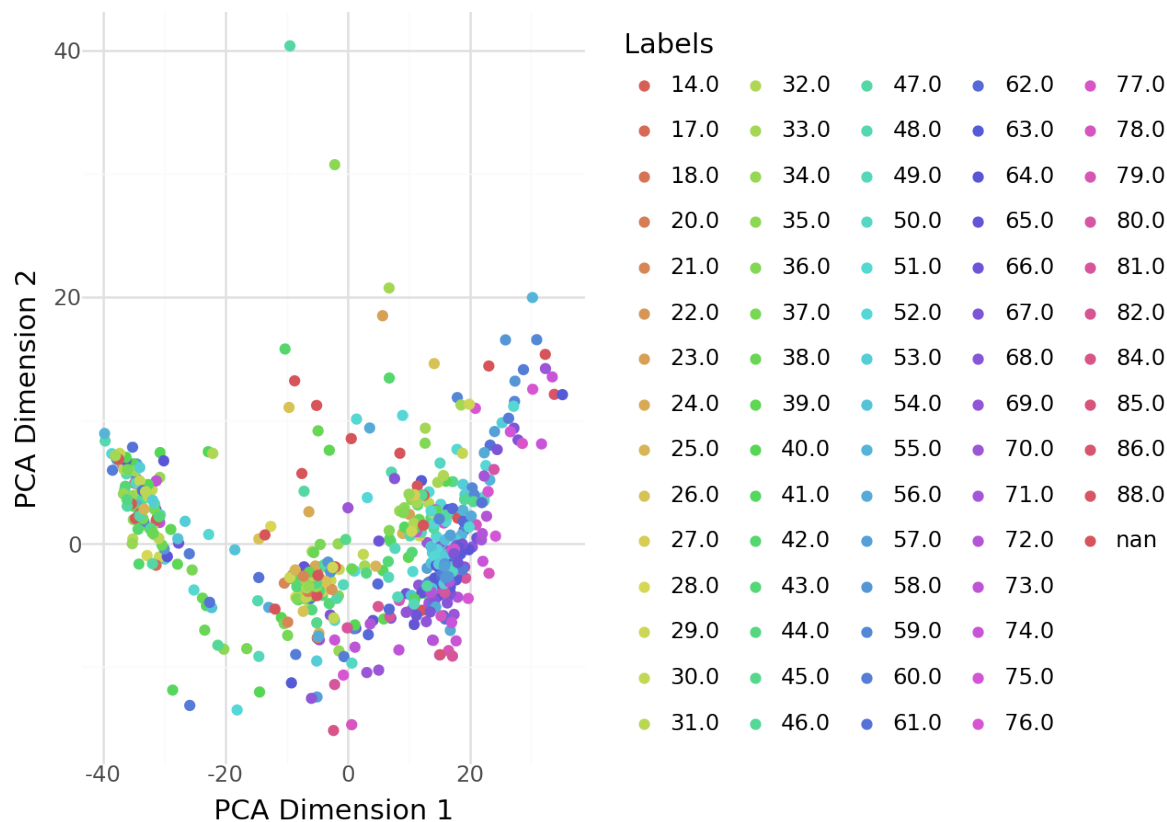
```
In [77]: train_embeddings = model.transform(train_dataset)
flexynesis.plot_dim_reduced(train_embeddings, train_dataset.annotation['OS_STATUS'])
```

PCA Scatter Plot with Colored Labels

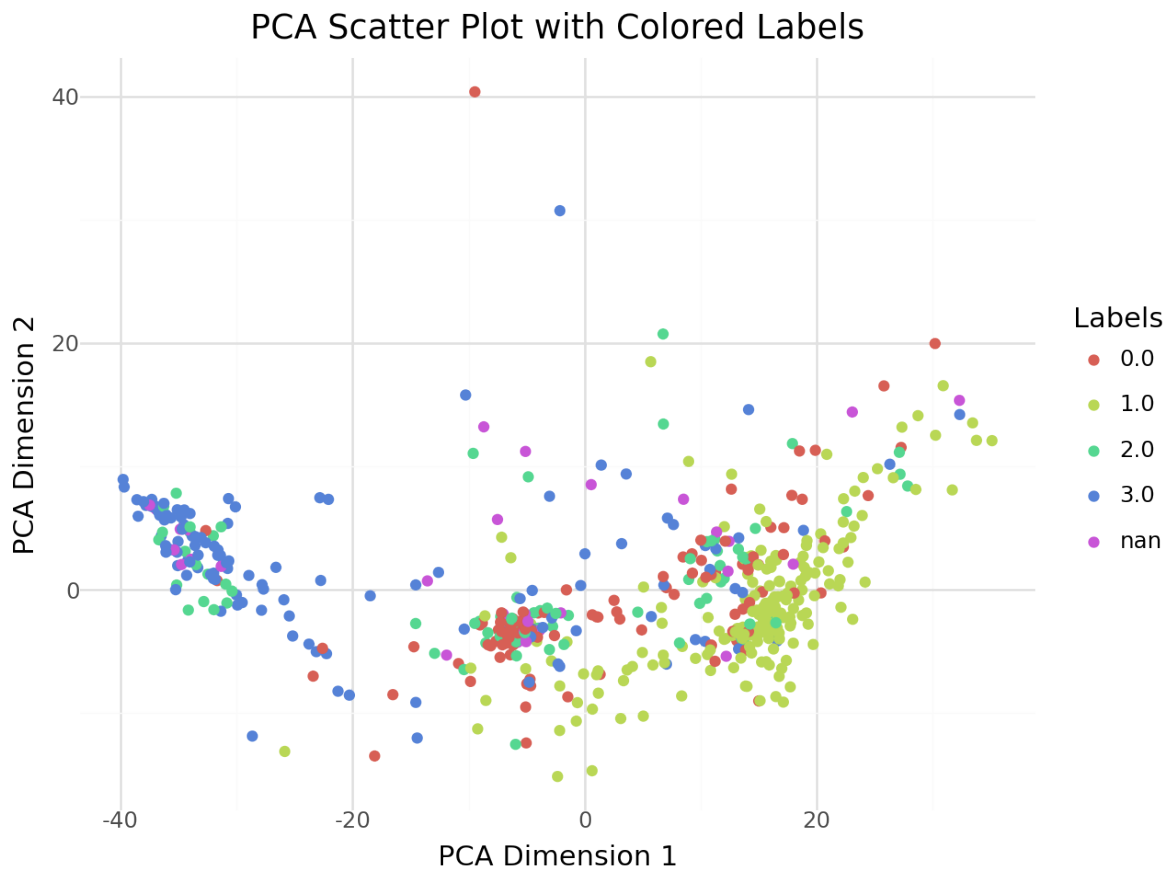


```
In [78]: train_embeddings = model.transform(train_dataset)
flexynesis.plot_dim_reduced(train_embeddings, train_dataset.ann['AGE'])
```

PCA Scatter Plot with Colored Labels



```
In [79]: train_embeddings = model.transform(train_dataset)
flexynesis.plot_dim_reduced(train_embeddings, train_dataset.ann['HISTOLOG
```



3.2 Survival-risk subtypes

Use the best model from the above exercises to inspect sample embeddings categorized by survival risk scores.

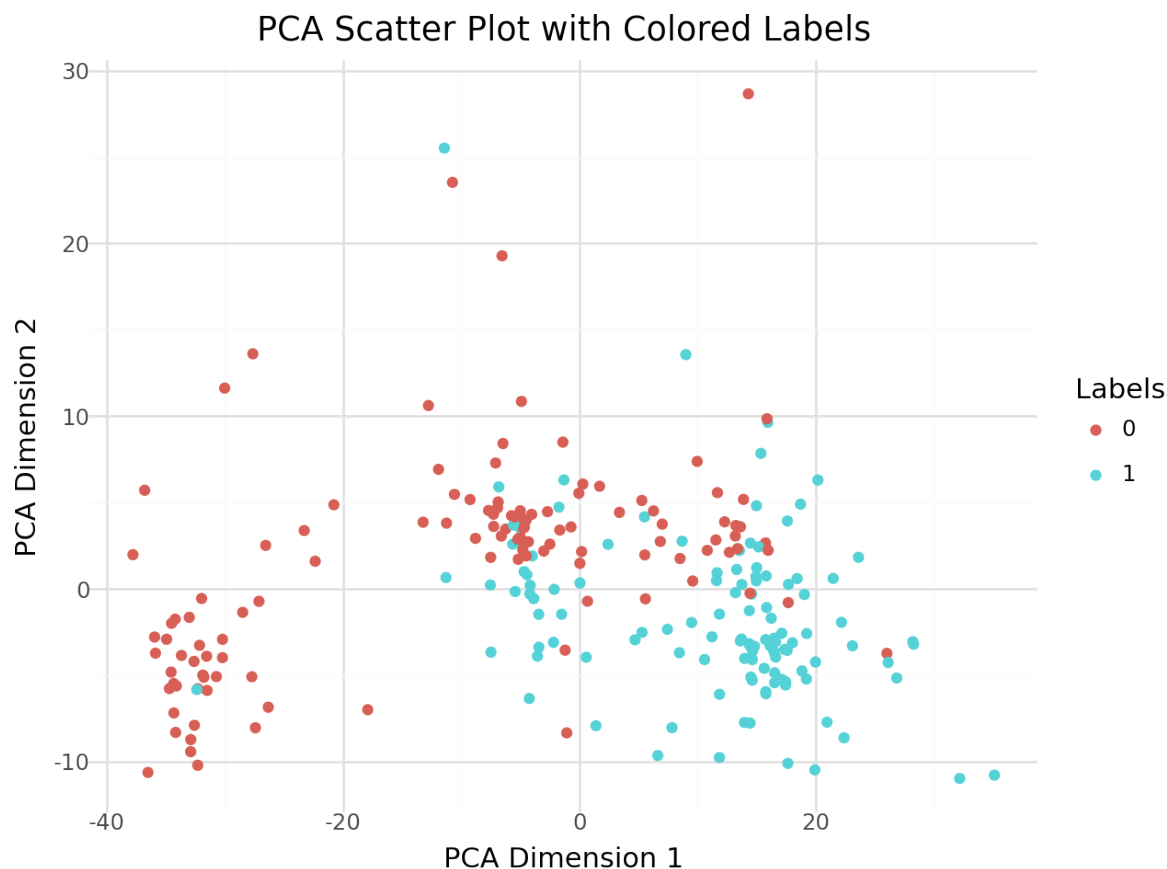
Let's group the samples by predicted survival risk scores into 2 groups and visualize the sample embeddings colored by risk subtypes.

Notice: You can use the code-below to get survival risk groups, however, notice that you must have built a model with "OS_STATUS" already.

```
In [80]: # get model outputs for survival variable
outputs = model.predict(test_dataset)['OS_STATUS'].flatten()
risk_scores = np.exp(outputs)
# Define quantile thresholds
quantiles = np.quantile(risk_scores, [0.5])
# Assign groups based on quantiles
groups = np.digitize(risk_scores, quantiles)
```

```
In [81]: # Extract sample embeddings
E = model.transform(test_dataset)
```

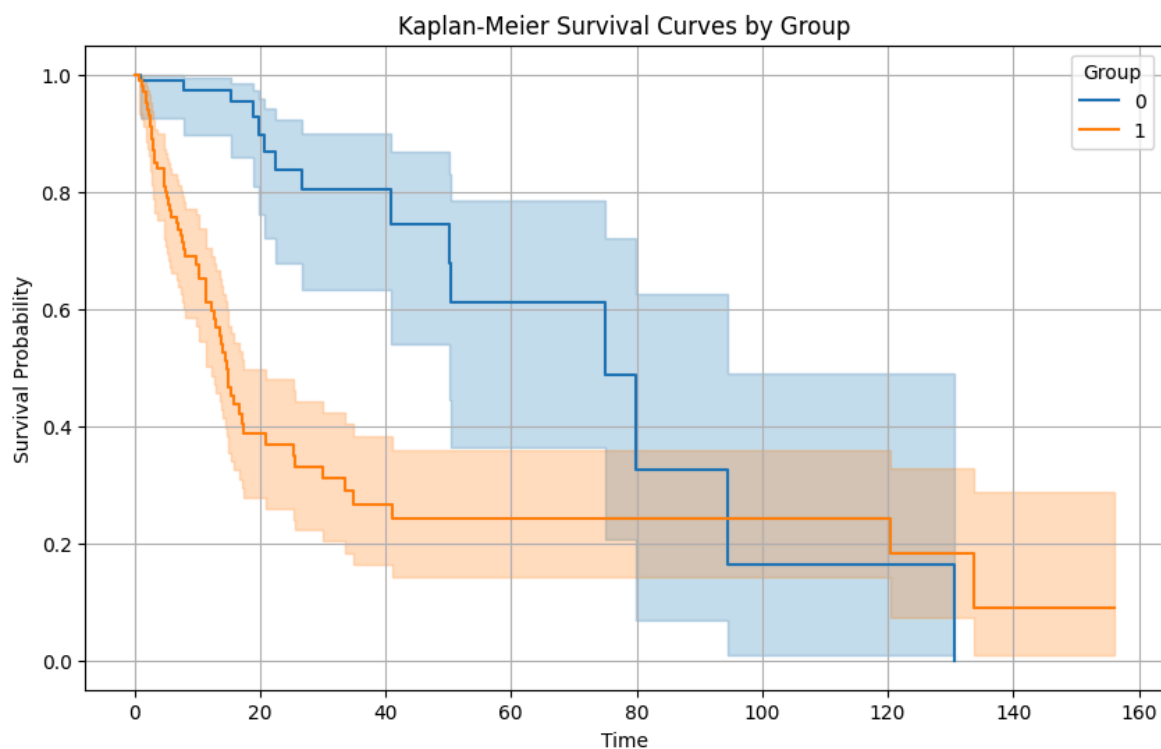
```
In [82]: flexynesis.plot_dim_reduced(E, groups)
```



Let's also see the Kaplan Meier Curves of the risk subtypes

```
In [83]: # remove samples with NA values first
durations = test_dataset.ann['OS_MONTHS']
events = test_dataset.ann['OS_STATUS']
valid_indices = ~torch.isnan(durations) & ~torch.isnan(events)
```

```
In [84]: flexynesis.plot_kaplan_meier_curves(durations[valid_indices], events[valid_indices])
```



Finding survival-associated markers

We can also compute feature importance scores for prediction of overall survival.

```
In [85]: model.compute_feature_importance(train_dataset, 'OS_STATUS')
```

```
In [86]: # get top 10 features
flexynesis.get_important_features(model, var = 'OS_STATUS', top=10)
```

```
Out[86]:
```

	target_variable	target_class	target_class_label	layer	name	importance
0	OS_STATUS	0		mut	IDH1	0.783055
1	OS_STATUS	0		mut	ATRX	0.273149
2	OS_STATUS	0		mut	NF1	0.100545
3	OS_STATUS	0		mut	IDH2	0.098217
4	OS_STATUS	0		cna	MIR603	0.092854
5	OS_STATUS	0		mut	CIC	0.063130
6	OS_STATUS	0		mut	MUC16	0.058835
7	OS_STATUS	0		mut	EGFR	0.054188
8	OS_STATUS	0		mut	TP53	0.050354
9	OS_STATUS	0		mut	PIK3CA	0.038126

Comparing top markers with clinical covariates

Let's build a linear Cox-PH model including the top 5 markers and other clinical variables such as histological diagnosis, disease type (STUDY), age, and sex.

```
In [87]: # define a data.frame with clinical covariates and top markers along with
vars = ['AGE', 'SEX', 'HISTOLOGICAL_DIAGNOSIS', 'STUDY', 'OS_MONTHS', 'OS_STATUS']
# read clinical variables
df_clin = pd.concat(
    [pd.DataFrame({x: train_dataset.ann[x] for x in vars}, index=train_dataset.index),
    pd.DataFrame({x: test_dataset.ann[x] for x in vars}, index=test_dataset.index)],
    axis = 0)
# get top 5 survival markers and extract the input data for these markers
imp = flexynesis.get_important_features(model, var = 'OS_STATUS', top=5)
df_imp = pd.concat([train_dataset.get_feature_subset(imp), test_dataset.get_feature_subset(imp)], axis=0)

# combine markers with clinical variables
df = pd.concat([df_imp, df_clin], axis = 1)
# remove samples without survival endpoints
df = df[df['OS_STATUS'].notna()]
df
```

Out [87]:

	cna_MIR603	mut_IDH1	mut_ATRX	mut_NF1	mut_IDH2	AGE	SEX	HIS
TCGA-HT-8113	0.542008	-1.018150	-0.585658	-0.267004	6.733003	49.0	0.0	
TCGA-14-0871	-1.218370	-1.018150	-0.585658	3.745267	-0.148522	74.0	0.0	
TCGA-HT-A618	0.593586	-1.018150	1.707482	-0.267004	-0.148522	37.0	0.0	
TCGA-DU-A7TD	-1.126427	-1.018150	-0.585658	-0.267004	-0.148522	52.0	1.0	
TCGA-HT-7485	0.577889	0.982173	1.707482	-0.267004	-0.148522	42.0	1.0	
...	...	...	...	...	...	...	...	...
TCGA-VM-A8CA	0.539766	0.982173	-0.585658	-0.267004	-0.148522	54.0	1.0	
TCGA-14-1043	-0.642042	-1.018150	-0.585658	3.745267	-0.148522	61.0	1.0	
TCGA-02-2470	0.546493	-1.018150	-0.585658	-0.267004	-0.148522	57.0	1.0	
TCGA-DU-6407	0.546493	0.982173	1.707482	-0.267004	-0.148522	35.0	0.0	
TCGA-27-1832	-0.668953	-1.018150	-0.585658	-0.267004	-0.148522	59.0	0.0	

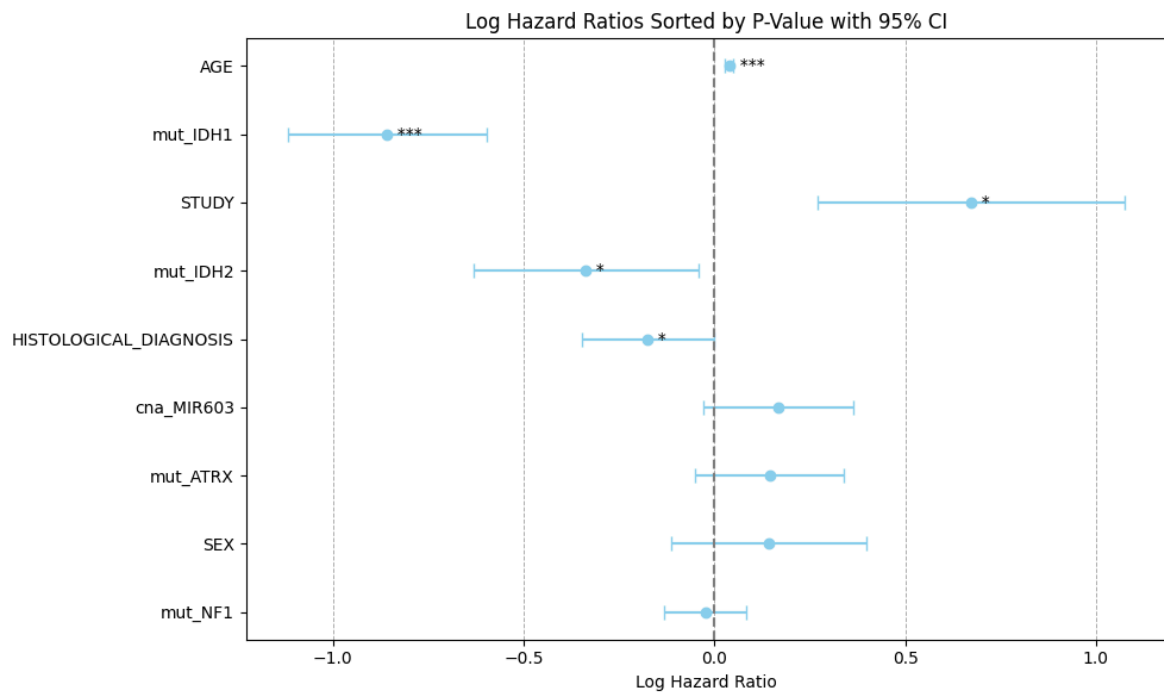
729 rows × 11 columns

```
In [88]: # build a cox model
cox = flexynesis.build_cox_model(df, 'OS_MONTHS', 'OS_STATUS')
```

No low variance features were removed based on event conditioning.

```
In [89]: # visualize log-hazard ratios sorted by p-values
flexynesis.plot_hazard_ratios(cox)
```

```
/usr/local/lib/python3.11/site-packages/flexynesis/utils.py:764: FutureWarning: Series.__getitem__ treating keys as positions is deprecated. In a future version, integer keys will always be treated as labels (consistent with DataFrame behavior). To access a value by position, use `ser.iloc[pos]`
```



3.3 Final Exercise

- Inspect the top 10 markers from section 3.2 and see if they have been characterized in the literature as important markers for Glioma disease progression.

```
In [90]: important_features = flexynesis.get_important_features(model, var='OS_STA
```

```
In [91]: important_features_df = pd.DataFrame(important_features)
```

```
In [92]: important_features_sorted = important_features_df.sort_values(by='importa
print(important_features_sorted)
```

	target_variable	target_class	target_class_label	layer	name	importan
0	OS_STATUS	0		mut	IDH1	0.7830
1	OS_STATUS	0		mut	ATRX	0.2731
2	OS_STATUS	0		mut	NF1	0.1005
3	OS_STATUS	0		mut	IDH2	0.0982
4	OS_STATUS	0		cna	MIR603	0.0928
5	OS_STATUS	0		mut	CIC	0.0631
6	OS_STATUS	0		mut	MUC16	0.0588
7	OS_STATUS	0		mut	EGFR	0.0541
8	OS_STATUS	0		mut	TP53	0.0503
9	OS_STATUS	0		mut	PIK3CA	0.0381

- **IDH1** ([Nobusawa et al, 2009](#))
- **ATRX** ([Koschmann et al, 2016](#))
- **NF1** ([Marques et al, 2021](#)) & ([Packer et al, 2020](#))
- **MIR603** ([Ramakrishnan et al, 2020](#))
- **CIC** ([Bunda et al, 2019](#)) & ([Darabi et al, 2023](#))
- **MUC16** ([V P Ferrer, 2023](#))

... These markers have been associated with glioma in the literature, and it is suggested that mutations or expression levels of these genes may have an impact on the progression of the disease.